GUIDE 29 · EVENTS & EXPORTS

Give each user of your app their own private activity feed

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=29-per-key-live-feeds. This copy was made on 4 November 2026.

Time: 15 minutes. **You'll end up with:** a live stream per user - in a mobile app, say - showing that user's own queries completing, and nobody else's, verified with two users side by side.

> **Stable** - live events are under the compatibility promise since 0.34 (what that means).

The rule that makes this work: **a stream shows only the runs made with the same credential that opened it.** The admin key sees everything; any other key - or signed-in user - sees only its own runs. So "one credential per user" gives "one private feed per user", with no filtering code of your own.

For how to parse the stream and reconnect, see Build a client that watches queries run. This guide is about who sees what.

Step 1: Run the events server

For an app with many users, use the dedicated events process rather than the main server's `/events` - it holds thousands of streams, sees runs from every instance, and lets a client resume where it left off:

```shell
QUERYAPIGATE_API_KEY=demo-key queryapigate events --port 5002
```

Run it beside `queryapigate serve`, against the same `QUERYAPIGATE_HOME` (or `QUERYAPIGATE_DATABASE_URL`). Events reach it from the event log in the store, within `QUERYAPIGATE_HISTORY_FLUSH_INTERVAL` (default 1 second); `0.2` makes them snappier. In production, route `/events` to it from your reverse proxy so the app keeps one base URL - see DEPLOYMENT.md.

The main server's own `GET /events` works the same way for a handful of streams (four by default), which is plenty for the Console and a few dashboards.

Step 2: One credential per user

Two ways. Both were verified in the run below.

A. One API key per user, created from a role when the user signs up:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/roles -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"name": "app-user", "connections": [], "queries": ["orders_by_status"]}'

curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"name": "app-alice", "role": "app-user"}'
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"name": "app-bob", "role": "app-user"}'
```

Your backend stores each user's secret and hands it to their app. Good for a few hundred users; beyond that you're managing a lot of keys.

B. Signed-in users (JWT) - no keys at all. If your app already has a login, QueryAPIGate can accept its tokens:

```shell
QUERYAPIGATE_JWT_SECRET=<at least 32 bytes> QUERYAPIGATE_JWT_ROLE=app-user queryapigate serve
QUERYAPIGATE_JWT_SECRET=<the same> QUERYAPIGATE_JWT_ROLE=app-user queryapigate events
```

Each user calls with `Authorization: Bearer <their token>` and is known as `jwt:<sub>`. For an identity provider (Auth0, Cognito, Keycloak …) use `QUERYAPIGATE_JWT_JWKS_URL` instead - see Signed-in users (JWT). Start both processes with the same JWT settings.

Step 3: What each one sees

Four streams open at once - alice's key, bob's key, the admin key, and carol's JWT - then five runs: alice, bob, alice, carol (JWT) and dave (JWT, no stream open). Verified result:

STREAM	EVENTS RECEIVED
alice (`app-alice`)	1 alice, 3 alice
bob (`app-bob`)	2 bob
carol (JWT)	4 `jwt:carol`
admin	1, 2, 3, 4, 5 - everyone, dave included

Each user's app sees only its own activity. Nothing in the request chooses the filter - it comes from the credential, so a user can't ask for someone else's feed.

```text
id: 1
data: {"type": "query.run", "source": "rest", "occurred_at": "2026-10-13 18:33:37", "kind": "saved",
       "query": "orders_by_status", "version": 1, "connection_name": "shop",
       "entry": {"executed_at": "2026-10-13 18:33:37", "key_name": "app-alice", "transport": "rest",
                 "status": "success", "rows": 3, "duration_ms": 1, ...}}
```

Step 4: Resume after a dropped connection

Phones lose connections. Send the last `id:` you received as `Last-Event-ID` when reconnecting, and the stream first replays what you missed - still only your own runs - then continues live. Verified: alice reconnected with `Last-Event-ID: 1` and received exactly `3`, nothing of bob's or carol's in between.

```text
GET /events
X-API-Key: <alice's secret>
Last-Event-ID: 1
```

The events server keeps each event for `QUERYAPIGATE_EVENTS_RETENTION_HOURS` (24 by default): a phone away for longer gets the stream from the oldest event still kept. Raise it if your users' phones sleep for longer.

Step 5: Ask for less

A stream can narrow what its credential already sees: `?type=`, `?kind=` (`saved` or `adhoc`), `?query=`, `?connection=` and `?status=` (`success` or `error`), combined. A filter never widens what a credential sees - alice asking for `?query=orders_by_status` still gets only her own runs. An operations screen that only wants failures:

```text
GET /events?status=error
X-API-Key: <the admin key>
```

Verified, in the same run: a query against a table that had been dropped failed (`500 query_failed`), and this stream received only that run - id 6, `old_orders`, `"status": "error"` - and none of the five that succeeded. A filter that can never match is refused rather than left silent: `?status=failed` answers `400`, `invalid_filter`.

Things to know

- **The feed is private; the data isn't automatically.** A feed shows a user their own *runs*. Whether the *rows* a query returns belong to that user is up to the query. With JWT, a parameter with `"from_claim": "sub"` fills in the user's id from their verified token, so `WHERE customer_id = :customer_id` can only ever return their own rows - see Signed-in users (JWT).
- **Credentials go in headers, never the URL.** A browser's `EventSource` can't set headers; read the stream with `fetch()` instead (guide 28 shows how). Native HTTP clients on iOS and Android can set headers on a streaming request.
- **Revoking a key ends its stream** within about a minute; an expired JWT ends it too - the app reconnects with a fresh token and `Last-Event-ID`.
- **A slow reader is cut off** once 1,000 events are waiting for it, and catches up by resuming.

Next steps

- Build a client that watches queries run - parsing and reconnecting.
- Create a role - the template behind per-user keys.
- Put QueryAPIGate behind a reverse proxy - routing `/events`, unbuffered.